4.2 Algorithm Details

We first present an algorithm *min Power Parity* (mPP) in Fig. 5 to place the VMs on the given set of servers in a manner such that the overall power consumed by all the servers is minimized. The algorithm takes as input the VM sizes for the current time window that can meet the performance constraints, a previous placement and the power model for all the available servers. It then tries to place the VMs on the servers in a manner that minimizes the total power consumed.

```
algorithm mPP
Input : ∀iVM_i, Alloc_old  Output = Alloc_new
 ∀Server_j
    Alloc_j = φ, Used_j = 0
 Sort VMs by size in decreasing order
 for i = 1 to N
    ∀Server_j compute Slope(Used_j)
    Pick the Server_min with the least Slope
    Add VM_i to Alloc_min, Used_min += Size(VM_i)
 End For
 Alloc_new = FFD(Used)
 returnAlloc_new
end mPP
```

Fig. 5. Power-minimizing Placement Algorithm

mPP works in two phases: In the first phase, we determine a target utilization for each server based on the power model for the server. The target utilization is computed in a greedy manner, where we start with a utilization of 0 for each server. We then pick the server with the least power increase per unit increase in capacity. We continue the process till we have allocated capacity to fit all the VMs. Since we may not be able to estimate the server with the least slope for all possible background traffic mixes, we pick an arbitrary traffic mix to model the power for each application and use this model in the selection process. We will later show that modeling based on an arbitrary background traffic mix also leads to a good solution. In the second phase, we call the bin-packing algorithm FFD based on *First Fit Decreasing* to place the VMs on the servers, while trying to meet the target utilization on each server. The bins in our version have unequal capacity, where the capacity and order of each bin is defined in the first phase whereas standard FFD that works with randomly ordered equal-sized bins.

Theorem 1. *If the* Ordering Property *or the* Background Independence Property *hold for a given set of servers and applications, then the allocation values obtained by* mPP *in its first phase are locally optimal.*

Proof. Let there be any two servers S_j and S_k such that we can shift some load between the two. For simplicity, assume that the load shift requires us to move